

AgriSeva-AI

Project Bible

A multilingual AI advisory platform for Indian agriculture

Owner: Atul Saini

Audience: Stakeholders, reviewers and new contributors

Generated: September 2026

Version: 1.0 · COMPREHENSIVE EDITION

CONFIDENTIAL — This document is the sole property of the project owner.

1. Executive Summary

AgriSeva-AI is a multilingual agricultural advisory platform purpose-built for Indian farmers. It accepts questions in twenty-two Indian languages — by text or voice — and returns reliable, context-aware answers drawn from three sources, in order of authority.

- Expert-verified answers stored in the Golden Dataset (MongoDB Atlas Vector Search).
- Standardised Package-of-Practices (PoP) guidelines for the relevant crop and state.
- AI-generated responses from a LangGraph orchestrator using Claude (Anthropic), self-hosted MiniMax, or self-hosted Gemma when no curated answer exists.

The platform is engineered as a three-tier system: a Vite + React frontend, a Node.js + Express backend, and a Python + LangGraph AI agent that fans out to eleven independent MCP tool servers (weather, market prices, soil, schemes, chemicals, knowledge base, location, and several more). Auth is handled by Firebase; data is stored in MongoDB Atlas; voice calls are powered by Plivo; translations and speech-to-text are handled by Sarvam AI.

Beyond chat, AgriSeva-AI ships a full operations console: a Farmer Dashboard with live mandi prices, a buyer directory, lot creation and offer workflow; a moderator and reviewer console for the human-in-the-loop answer quality pipeline; a coordinator console for field staff; an admin console; and observability for the entire system. The same AI agent is reachable from a WhatsApp number and an inbound Plivo voice line, so farmers can use whatever channel they already trust.

Highlights as of this edition

- 28 authenticated frontend routes + a public landing/auth flow.
- 23 backend modules, 250+ REST endpoints, all JWT-guarded by Firebase Admin SDK.
- 1 supervisor agent + 6 subagents + 11 MCP tool servers (FastMCP).
- 22 Indian language packs for the entire UI, with graceful English fallback.
- Farmer Dashboard with a 13-route surface including live mandi prices (Agmarknet + eNAM).
- Auth & persistence hardening complete on farmer/transaction modules — 18 new backend tests passing.
- Modern responsive shell: sidebar is now toggleable on every viewport (mobile, tablet, desktop).

Recent milestones

Milestone	Status	Evidence
Farmer Dashboard real-data...	' Complete	FARMER_DASHBOARD_DATA_A...
Auth + persistence hardening (x-...	' Complete	FARMER_SYSTEM_AUTH_PER...
Sidebar toggleable on mobile/...	' Complete	Frontend HMR confirmed via docs/.
i18n coverage expansion to 22...	' Complete	frontend/src/locales/*
Cloud Run Jobs migration for...	' Complete	.github/workflows/cloudrun-jobs-...
Deployment readiness audit +...	' Complete	DEPLOYMENT_READINESS.md
Payment-gateway integration	& Deferred	Records without real gateway
Real buyer KYC	& Deferred	Seeded catalog only

2. Stakeholders & User Roles

AgriSeva-AI recognises eleven role strings on the wire (see `frontend/src/lib/roles.ts`). Each maps to a distinct mix of navigation, API permissions, and AI tool surface. The platform uses Firebase Auth as the identity provider and a JWT trust chain to the backend.

Role	Code	Primary Job-to-be-Done	Surface
Admin	admin	Operate the platform, manage roles, full audit access	Admin console
Moderator	moderator	Triage incoming questions, allocate reviewers	Moderator console
Expert / PAE-Expert	expert / pae_expert	Write and review answer content for the Golden Dataset	Reviewer console
Gate Keeper / Auditor	gate_keeper / auditor	First-pass and second-pass quality gating	Quality control console
Reviewer	reviewer	Approve or rewrite AI-generated answers	Reviewer console
Tester	tester	Run controlled flows during pre-release	Same as moderator
Call Agent	call_agent	Operate the inbound voice / WhatsApp queue	Chatbot console
District Coordinator	district_coordinator	Field operations for a district	/coordinator
Block Coordinator	block_coordinator	Field operations for a block	/coordinator
Village Volunteer	village_volunteer	Field operations for a village	/coordinator
Farmer	farmer	Ask questions, manage lots and offers, view prices	/farmer dashboard

Role helpers (`frontend/src/lib/roles.ts`)

- `COORDINATOR_ROLES` — `district_coordinator`, `block_coordinator`, `village_volunteer`.
- `USER_MANAGEMENT_ROLES` — `admin`, `moderator`, `tester`, `gate_keeper`, `auditor`.
- `QUEUE_DETAILS_ROLES` — `admin`, `moderator`, `gate_keeper`, `auditor`.
- `isCoordinatorRole(role)` and `canManageUsers(role)` are the two predicates used by the global nav.

What each role sees on sign-in

Role	Brand-Link Home	Visible Global Nav	Sub-Console
admin	/home	Dashboard · WhatsApp History · ChatBot Analytics · Notifications...	Admin tools + review
moderator	/home	Dashboard · WhatsApp History · ChatBot Analytics · Notifications...	Moderator queue
expert	/home	Dashboard · WhatsApp History · Notifications · Submission History	Reviewer console
pae_expert	/pae-expert	WhatsApp History · Notifications	PAE-Expert drafting
gate_keeper / auditor	/home	Dashboard · WhatsApp History · Notifications · Audit · Flags...	Audit console
reviewer	/home	Dashboard · WhatsApp History · Notifications	Reviewer console
call_agent	/home	Dashboard · Notifications	Call console
district/block/village	/coordinator	Coordinator-only (nav suppressed)	Field ops console
farmer	/farmer	Farmer Dashboard · Notifications	Farmer dashboard

3. High-Level Architecture

AgriSeva-AI is composed of four logical services. Each service can be deployed independently; in production they are isolated networks connected through Tailscale and a reverse proxy.

3.1 The four services

Frontend (browser)	Vite + React 19 + TypeScript. Static SPA, served by nginx in Docker, Firebase Hosting in prod.
Backend (API)	Express 5 + routing-controllers + InversifyJS. REST + JSON, JWT-validated by Firebase Admin SDK.
AI Agent (reasoning)	FastAPI + LangGraph supervisor + 6 subagents. Standard LangGraph HTTP+SSE.
MCP Tool Servers	11 FastMCP servers, each one a domain (weather, market, soil, schemes, ...). Accessed by the agent via MCP-over-HTTP, not by import.

3.2 Request flow — a farmer question

- Farmer texts the WhatsApp number, speaks to the Plivo line, or opens the web UI.
- Frontend (or WhatsApp/Plivo service) opens an HTTP/SSE stream to the AI Agent.
- Supervisor agent classifies the query and dispatches to one or more subagents.
- Subagents call MCP tools (Agmarknet for mandi prices, IMD for weather, ...) or hit RAG.
- Synthesizer returns a draft answer; reviewer system logs it for the human-in-the-loop.
- Same synthesizer path is used by the reviewer console and the WhatsApp-bot.

3.3 Logical diagram



```
| MCP SERVERS · 11 services · FastMCP |
| agmarknet · eNAM · IMD weather · soil · POP · GDB · schemes · chemicals ... |
+-----+

```

4. Technology Stack

Layer	Technology	Purpose
UI framework	React 19 + Vite 6	Single-page app with fast HMR and small...
UI primitives	Radix UI + shadcn/ui	Accessible, headless components
Styling	TailwindCSS 4	Utility-first CSS
Motion	Framer Motion + tailwindcss-animate	Transitions, drawer animations
Routing	TanStack Router (file-based)	Type-safe routes, code-splitting
Server state	TanStack Query	Cache + retries for REST calls
Client state	Zustand	Lightweight, no boilerplate
Forms / Validation	Zod + react-hook-form	Schema-validated forms
Auth (client)	Firebase Auth SDK	Google + email/password
Charts	Recharts	Trends inside dashboards
Map	react-leaflet	Geo-aware views in the farmer dashboard
Voice browser SDK	Plivo Web SDK	Browser SIP for callers
i18n	Custom i18n hook + 22 locale files	Indian languages
Toast	sonner + react-hot-toast	Two libraries for redundancy
Backend HTTP	Express 5 + routing-controllers +...	Decorators, DI
Backend TS	TypeScript (ESM)	Strict mode
Backend auth	Firebase Admin SDK	JWT verification per request
Backend jobs	node-cron + Google Cloud Run Jobs	Schedules
Backend WS	ws	Voice streaming
API docs	Scalar	Auto-generated from decorators
AI runtime	Python 3.11 + FastAPI + LangGraph	Streaming agent
LLM (primary)	Claude Sonnet (Anthropic)	Reasoning
LLM (self-hosted)	MiniMax + Gemma	Specialised subagents
Embeddings	BAAI/bge-large-en-v1.5 (self-hosted)	Vector retrieval
Vector DB	MongoDB Atlas Vector Search	Semantic search
Translation / STT	Sarvam AI	Indian languages
Voice (telephony)	Plivo	PSTN + WebRTC
Chat	WhatsApp Cloud API	Farmer chat
Storage	Google Cloud Storage	Backups, file attachments
Observability	Sentry	Errors and performance
AgriSeva-AI · Project Bible · Confidential CI	GitHub Actions	Build / test / deploy
Containers	Docker + Docker Compose	Local dev + Portainer VM

5. Repository Layout

The repository is a monorepo of four logical services plus shared types. Working directory: C:\Users\saini\OneDrive\Desktop\CSP\AgriSeva-Ai

```
AgriSeva-Ai/
|-- frontend/                                # Vite + React 19 + TypeScript
|   |-- src/
|   |   |-- routes/                         # TanStack Router file-based routes
|   |   |-- components/                    # MainDashboardShell/Sidebar/Header/Nav
|   |   |-- features/                      # Per-product-area code (farmerDashboard, etc.)
|   |   |-- hooks/api/                    # Auto-generated TS client + custom wrappers
|   |   |-- locales/                      # 22 language packs
|   |   |-- lib/                          # utils, roles, env config
|   |   |-- services/                     # Firebase, push, websockets
|   |   |-- stores/                       # Zustand stores
|   |   |-- mocks/                        # MSW handlers for dev
|   |   |-- config/                       # Runtime / build config
|   |-- package.json
|-- backend/                                # Express 5 + routing-controllers
|   |-- src/
|   |   |-- index.ts                      # Server entry
|   |   |-- bootstrap/                   # IoC container, app factory, jobs
|   |   |-- modules/                     # 23 modules
|   |   |-- shared/                      # middleware, interfaces, errors
|   |   |-- config/                      # env-driven config
|   |   |-- workers/                     # queue workers
|   |   |-- utils/                       # logDetails, env
|   |   |-- jobs/                        # cron + Cloud Run jobs
|   |-- scripts/                          # LGD sync, DB migrations
|-- ai/                                    # Python 3.11 + FastAPI + LangGraph
|   |-- agriseva/agents/                  # 40+ files; agriseva.py + subagents + synthesis
|   |-- tests/                            # pytest, MCP connectivity, contracts
|-- mcp/                                  # FastMCP tool servers (11 services)
|-- docs/                                 # This bible + cloudrun-jobs-setup.md
|-- .github/workflows/                   # CI / deployment workflows
|-- docker-compose*.yaml                 # Local, pull, portainer stacks
|-- firebase.json
|-- DEPLOYMENT_READINESS.md
|-- DEPLOYMENT_SETUP.md
|-- FARMER_DASHBOARD_DATA_AUDIT.md
|-- FARMER_SYSTEM_AUTH_PERSISTENCE_HARDENING.md
|-- Complete_Project_Architecture_and_Feature_Documentation.pdf
```

6. Frontend — Vite + React + TypeScript

6.1 Stack & dev flow

- ``pnpm install`` from frontend/ to install.
- ``pnpm run dev`` runs Vite on port 5173 with HMR.
- ``pnpm run build`` produces the static bundle and runs TypeScript validation.
- ``pnpm run test`` runs Vitest suites.
- ``pnpm run openapi-ts`` regenerates the typed API client from the backend's OpenAPI.

6.2 File-based route map (28 authenticated + public)

Routes live in `frontend/src/routes/` and follow the TanStack Router convention ``_authenticated.<segment>.tsx``. The shell enforces auth and renders the global nav.

Route	File	Roles	Purpose
/	_root + index.tsx	public	Landing
/auth	_auth.tsx	public	Sign-in / regi
/home	_authenticated.home.tsx	all except farmer/pae/...	Main dashbo
/farmer	_authenticated.farmer.tsx	farmer	Farmer sub-l
/farmer (home)	_authenticated.farmer.index.tsx	farmer	Farmer dash
/farmer/prices	_authenticated.farmer.prices.tsx	farmer	Live mandi p
/farmer/buyers	_authenticated.farmer.buyers.tsx	farmer	Buyers direc
/farmer/buyers/:buyerId	_authenticated.farmer.buyers.\$buyerId.tsx	farmer	Buyer detail
/farmer/lots	_authenticated.farmer.lots.tsx	farmer	My lots
/farmer/lots/new	_authenticated.farmer.lots.new.tsx	farmer	Create a lot
/farmer/lots/:lotId	_authenticated.farmer.lots.\$lotId.tsx	farmer	Lot detail + c
/farmer/offers	_authenticated.farmer.offers.tsx	farmer	All offers
/farmer/payments	_authenticated.farmer.payments.tsx	farmer	Transactions
/farmer/grievances	_authenticated.farmer.grievances.tsx	farmer	Support griev
/farmer/storage	_authenticated.farmer.storage.tsx	farmer	Storage book
/farmer/logistics	_authenticated.farmer.logistics.tsx	farmer	Logistics bo
/farmer/profile	_authenticated.farmer.profile.tsx	farmer	Farmer profil
/farmer/recommend	_authenticated.farmer.recommend.tsx	farmer	Compare buy
/whatsapp-history	_authenticated.whatsapp-history.tsx	except call_agent	Conversation
/chatbot	_authenticated.chatbot.tsx	admin / moderator	ChatBot ana
/notifications	_authenticated.notifications.tsx	all	Notification c
/audit	_authenticated.audit.tsx	admin/auditor/...	Audit trail
/flags-reported	_authenticated.flags-reported.tsx	admin/mod/auditor/...	Flagged que

6.3 Global layout (the four pillars)

The authenticated experience is composed of exactly four layout components. Together they form a single responsive shell that is identical from any starting route.

- `MainDashboardShell` — orchestrates the sidebar drawer on mobile, the collapsible sidebar rail on desktop, the top header bar and the main content slot.
- `MainDashboardSidebar` — the persistent vertical nav for ≥ 1024 px and the slide-in drawer for mobile/tablet; collapses to a 68-72px rail with icon-only tooltips.
- `MainDashboardHeader` — brand + primary horizontal nav ($\geq md$), notifications bell, profile menu, language switcher, hamburger that toggles the sidebar.
- `MainDashboardNav` — single source of truth for the visible nav list per role (`NAV_ITEMS`) and per-role `homeRouteFor(user)`.

Child layouts (e.g. `FarmerLayout`) are layered inside the global shell so the brand row, sign-out, profile menu and language switcher are provided exactly once for the whole app.

6.4 Recent UI work — Sidebar toggleable everywhere

- Before: persistent-only on desktop; sidebar and tab strip cramped on ≤ 1023 px.
- After: hamburger in the header toggles a slide-in drawer on mobile/tablet (≤ 1023 px) and collapses the persistent sidebar to a 68-72px rail on desktop with a width transition.
- Implementation: new dual-mode state in `MainDashboardShell.tsx`; icon button in `MainDashboardHeader` calls `sidebar.toggle()`; `MainDashboardSidebar` renders either the drawer variant or the persistent rail.
- Single nav source: `NAV_ITEMS` in `MainDashboardNav.tsx` is consumed by both sidebar and header.
- Role-aware first item: Farmer Dashboard is the first entry for users with `role=farmer`.
- `i18n`: drawer/expand/collapse labels fall back to English strings via `t(key, fallback)`; ready to be replaced with locale keys batch-wise.
- One-line fix in this iteration: a strict null narrowing on the `isCoordinatorRole` callback (`MainDashboardNav.tsx:118`) prevented the typecheck from passing — resolved with `!!u &&`.
- Local verification: Vite dev server (`127.0.0.1:5173`) returns HTTP 200 on `/`, `/farmer`, and the four modified components transpile cleanly.

6.5 State, data and `i18n`

- TanStack Query for all REST fetches; optimistic updates on offer/lot mutations.
- Zustand for ephemeral UI state (drawer open, current filter, language preference mirror).
- 22 language locales under `frontend/src/locales/*.ts` plus an English fallback baked into `t(key, fallback)`.
- MSW mocks under `src/mocks` for offline dev; gated by `VITE_ENABLE MOCKS`.
- Auto-generated TS client under `src/client` wraps every backend controller.

7. Backend — Express 5 + Routing-Controllers

The backend is a TypeScript Express 5 server that uses routing-controllers for declarative endpoints, InversifyJS for DI, and Firebase Admin for auth. It compiles to ESM and is run with ``node build/index.js`` in production.

7.1 Modules — 23 in total

#	Module	Responsibility
1	acc-agent	Human-in-the-loop ACC agent integration
2	ai	Proxy to the LangGraph AI agent over HTTP
3	answer	Answer creation, edits, versioning
4	auditTrails	Tamper-evident audit logs
5	auth	Firebase-based authentication, OAuth callback
6	chatbot	WhatsApp/voice chatbot integration
7	chemical	Chemical/pesticide registry + banned check
8	comment	Threaded comments on questions
9	context	Per-call context records
10	core	Core utilities, base controller
11	crop	Crop catalogue
12	dashboard	Analytics aggregations
13	lgd	LGD (states/districts/blocks/villages) sync
14	marketIntelligence	Agmarknet + eNAM price ingestion & REST
15	notification	Push / in-app notifications
16	performance	Reviewer performance metrics
17	plivo	Plivo voice call handling
18	question	Core Q&A review workflow
19	request	Generic request feedback
20	reroute	Question re-routing between reviewers
21	transaction	Lots, offers, payments, grievances, buyers, storage, logistics
22	user	Users, roles, farmer profile sub-doc
23	whatsapp	WhatsApp webhook

7.2 Per-module file layout (conventional)

- controllers/ — routing-controllers decorated classes (@JsonController, @Get, @Post).
- services/ — business logic, transactions, side effects.
- repositories/ — Mongo access (one repository per collection).
- validators/ — class-validator DTOs.
- tests/ — Vitest suites (controllers + services).
- transformers/ — class-transformer interceptors.

7.3 Cross-cutting concerns

- CORS driven by APP_ORIGINS env var; no other allow-lists.
- All authenticated endpoints use @Authorized() + @CurrentUser() from routing-controllers.
- Errors are normalised by a single Express error middleware in src/shared/middleware.
- All write paths log to auditTrails; reads log PII-safe summaries via utils/logDetails.ts.
- Generated OpenAPI spec is consumed by the frontend's openapi-ts to produce src/client/*.

7.4 Auth + persistence hardening on farmer/transaction

Recent (this edition) — see FARMER_SYSTEM_AUTH_PERSISTENCE_HARDENING.md:

Controller	Auth model	Ownership checks	Notification fan-out
LotController	JWT	user._id	lot_created / lot_updated
OfferController	JWT	lot owner check	offer_received / offer_accepted / offer_rejected
PaymentController	JWT	owner check	payment_recorded
GrievanceController	JWT	owner check	grievance_created
StorageController	JWT	owner check	storage_booked
LogisticsController	JWT	owner check	logistics_booked
BuyerController	JWT	owner check	(none)
UserController	JWT	self-only PATCH /users/me/...	n/a

- Removed x-demo-farmer-id trust header from every endpoint and frontend client.
- Added IFarmerProfile sub-doc + FarmerProfilePatchDto with merge semantics + array dedup.
- Added PATCH /api/users/me/farmer-profile for the editable profile UI.
- NotificationService now fires real events so the notifications bell mirrors backend truth.
- New test files: TransactionAuth.test.ts (9/9), UserProfile.test.ts (5/5), NotificationFlow.test.ts (4/4).
- Net diff: 15 files modified, +876 / -177 lines.

8. AI Agent — LangGraph Supervisor + Subagents

The AI Agent is a Python service: FastAPI for transport, LangGraph for orchestration, Anthropic Claude for the reasoning LLM, and self-hosted MiniMax / Gemma for special-purpose subagents. It is reachable over a standard LangGraph HTTP+SSE API so any client (WhatsApp, voice, ChatUI, future) can plug in unchanged.

8.1 Files

```
ai/agriseva/agents/
|-- agriseva.py           # Graph entrypoint + compiled `graph`
|-- state.py             # AgentState schema (Pydantic)
|-- planner.py           # Planner orchestrator (USE_PLANNER_GRAPH=true)
|-- planner_rules.py     # Rule-based short-circuits
|-- plan_executor.py     # Plan execution loop
|-- prompts.py           # System + per-agent prompts
|-- domains.py           # Domain tagging for routing
|-- config.py            # Model + endpoint config
|-- language.py          # Language detection + Sarvam invocation
|-- translate_answer.py  # Answer translation
|-- synthesis/           # Final answer assembly (12 files)
|-- subagents/           # GDB, weather, market, soil, schemes, chemicals
|-- support/            # memory, logging, trace, retrieval, registry
|-- tests/              # Unit tests for routing, planner, etc.
```

8.2 Status of every major capability (from ai/README.md)

Capability	Status	Notes
Supervisor + subagent pattern	' Done	Main agent orchestrates GDB, weather, market,...
Planner orchestrator	' Done & stable	USE_PLANNER_GRAPH=true (default); planner

8.3 Evaluation harness

- evaluation/run.py + evaluation/plan.py + evaluation/nodes.py + evaluation/routing.py + evaluation/stream_debug.py.
- evaluation/all_tools.py and tool.py list every MCP tool the harness can exercise.
- evaluation/langsmith_*.py and evaluation/deepeval_*.py integrate LangSmith + DeepEval.
- evaluation/questions.py + report.py + summary.py form the golden-dataset scoring suite.
- evaluation/failure.py captures regression diffs.

9. MCP Tool Servers

Eleven FastMCP servers run as independent processes/containers. The AI Agent reaches them at runtime through an MCP client over HTTP — never by import. This keeps each tool server's deployment independent and lets us add or replace tools without modifying the agent.

Server	Domain	Backing API / Data
agmarknet-mcp	Mandi prices (primary)	agmarknet.gov.in
market	Mandi prices (alt)	Agmarknet cache + eNAM
imd_weather	Weather	India Meteorological Department
weather	Weather (alt. harness)	OpenWeather + IMD
soil-health-mcp	Fertilizer dosage	soilhealth.dac.gov.in
pop	Package of Practices	KVK catalogues
pop_v2	PoP v2 (richer structure)	KVK catalogues
golden_dataset	RAG — expert answers	MongoDB Atlas Vector Search
faq	FAQ document lookup	Internal
review-search-mcp	Review history search	Reviewer console backend
reviewer_system	Reviewer workflow	Reviewer console backend
other_markets (unified)	Unified mandi price scraper	Agmarknet + eNAM

9.1 Tool calling pattern

The supervisor classifies a query, then dispatches to the relevant subagent(s). Each subagent is a dedicated LLM ReAct loop with access only to its own MCP server(s). The subagent returns a clean result to the supervisor, which passes it to the synthesizer for the final answer.

10. Database Schema

Two engines are in play: MongoDB Atlas for the operational + analytics data, and PostgreSQL (asyncpq) for LangGraph checkpoint state. Vector search uses MongoDB Atlas Vector Search over the `goldenDataset` collection. Each collection has a repository wrapper in the matching backend module.

10.1 MongoDB — collections

Collection	Owner module	Purpose
users	user	Firestore-anchored user records + farmerProfile...
questions	question	Incoming farmer questions
answers	answer	Approved answers
requestfeedbacks	request	User feedback on answers
contexts	context	Per-call context records
notifications	notification	In-app + push notification fan-out log
auditTrails	auditTrails	Append-only audit log
comments	comment	Threaded comments on questions
crops	crop	Crop master
chemicals	chemical	Chemical + banned registry
marketPriceSnapshots	marketIntelligence	Agmarknet + eNAM daily snapshots
lots	transaction	Farmer grain lots for sale
offers	transaction	Buyer offers on lots
payments	transaction	Payment transactions (records, no gateway)
buyers	transaction	Buyer catalogue (demo seed)
grievances	transaction	Farmer support tickets
storage_options	transaction	Storage facilities catalogue (seed)
storage_bookings	transaction	Storage bookings
logistics_options	transaction	Logistics provider catalogue (seed)
logistics_bookings	transaction	Logistics bookings
whatsappThreads	whatsapp	Per-conversation thread log
lgd_states districts blocks villages	lgd	India LGD sync tables
schemes	(MCP)	Government schemes catalogue
goldenDataset	(MCP)	Curated vector-search knowledge base

10.2 PostgreSQL — LangGraph checkpoint store

- Stores conversation state and agent memory.
- Configured via DATABASE_URL; reachable on the Docker network at ai-postgres:5432.
- Connection limit + pool tuning per deployment.

11. Localization — 22 Indian Languages

The UI ships with locale packs for 22 Indian languages. Every visible string is looked up via `useTranslation -> t(key, fallback)` so missing keys fall back to the English string embedded in the source. This makes the app readable end-to-end in every language, even when a specific key hasn't been translated yet.

ISO 639	Language	File	Status
en	English	en.ts	' Complete
hi	Hindi	hi.ts	' Complete
bn	Bengali	bn.ts	' Complete
ta	Tamil	ta.ts	' Complete
te	Telugu	te.ts	' Complete
kn	Kannada	kn.ts	' Complete
ml	Malayalam	ml.ts	' Complete
mr	Marathi	mr.ts	' Complete
gu	Gujarati	gu.ts	' Complete
pa	Punjabi	pa.ts	' Complete
or	Odia	od.ts	' Complete
ur	Urdu	ur.ts	' Complete
as	Assamese	as.ts	' Complete
mai	Maithili	mai.ts	' Complete
sd	Sindhi	sd.ts	' Complete
ks	Kashmiri	ks.ts	' Complete
doi	Dogri	doi.ts	' Complete
kok	Konkani	kok.ts	' Complete
mni	Manipuri (Meitei)	mni.ts	' Complete
sa	Sanskrit	sa.ts	' Complete
sat	Santali	sat.ts	' Complete
brx	Bodo	brx.ts	' Complete

Note on the brand new sidebar/header labels: the `i18n` keys (`dashboard.openMenu`, `closeMenu`, `expandNav`, `collapseNav`, `globalNav`, `tabs*`) are added on demand by the UI's `t()` with English fallback strings, so the UI never breaks before the next batch translation pass.

12. Authentication & Authorization

Firebase Auth is the single identity provider for web users. The WhatsApp/PSTN channels issue their own tokens that the backend exchanges for short-lived session JWTs. All API endpoints (except /api/health, /api/auth/* and the OpenAPI doc) require a valid bearer token.

12.1 JWT verification

- Firebase Admin SDK validates the Authorization: Bearer <token> header on every protected route.
- Token claims (uid, email, email_verified, custom:role) populate req.user.
- routing-controllers' @CurrentUser() decorator exposes the principal.
- @Authorized() is used on every controller method that needs auth; ownership checks are added inline.

12.2 Cross-user authorisation

Failure	Returned status	Reason
Missing / invalid / expired token	401 Unauthorized	No principal resolved
Authenticated but wrong owner	403 Forbidden	Cross-user access attempt
Disabled / disallowed method	405 Method Not Allowed	Express default
Malformed body / validation failure	400 Bad Request	class-validator pipeline

12.3 Auth for the WhatsApp / Plivo channels

- WhatsApp channel uses the channel-internal user id from the chatbot module's mapping table.
- Plivo voice channel issues a per-call short-lived JWT that expires after call end.
- INTERNAL_API_KEY env var guards service-to-service calls (e.g. AI agent -> backend audit logging).

13. Communication Channels

13.1 Plivo — browser & PSTN voice

- Plivo Web SDK in the browser for outbound SIP calling from the dashboard.
- Plivo PSTN numbers for inbound voice to the AI agent — recorded, sent through STT and answered by the LangGraph agent.
- Plivo credentials in frontend `VITE_PLIVO_*`; backend `plivo/` module handles DTMF + hangup.
- Streaming audio is supported through a custom `ws / WebSocket` bridge.

13.2 WhatsApp

- WhatsApp Cloud API integration in `backend/src/modules/whatsapp/`.
- Per-conversation logs persisted in `whatsappThreads`.
- Same LangGraph agent as the chat UI — only the transport differs.
- Outbound notifications flow through the same channel.

13.3 Push notifications

- Web push using VAPID public key in `VITE_VAPID_PUBLIC_KEY` + browser-side `PushManager` subscription.
- Backend `notification/` module fans out from any domain event into both the bell icon and push.
- Bell icon shows the authenticated user's persisted notifications via `GET /api/notifications`.

14. Surface-by-Surface Feature Inventory

This section enumerates every user-facing capability for every persona. For the Farmer dashboard, the data source verdict column comes from the live smoke-test audit (FARMER_DASHBOARD_DATA_AUDIT.md).

14.1 Admin & Moderator

Feature	Where	Notes
User management	/user/:userId	List, search, change role, ban.
Submission history	/history	Filterable history of every submission.
Per-user history	/user/:userId/history	Drilldown of an individual user's work.
Audit trail	/audit	Tamper-evident log viewer.
Flags reported	/flags-reported	Triages flagged questions.
ChatBot analytics	/chatbot	Volume, latency, sentiment over time.
Notifications	/notifications	System alerts.
Profile menu	Header	Switch language, sign out.

14.2 Reviewer / Expert / PAE-Expert

- /pae-expert — full-page authoring surface for PAE experts (draft, save, request review).
- Reviewer console (pae-expert-page feature) — read incoming AI drafts, accept, rewrite, or reject.
- /home (Submission history) for experts to look up their own past submissions.
- Approval pipeline is human-in-the-loop: every AI-generated answer that goes out is also enqueued for expert review.

14.3 Gate Keeper / Auditor

- /audit — full audit trail with per-action filters.
- /flags-reported — flagged questions queue.
- Quality console (gate-keeper auditor queue) — automated via gateKeeperAuditorQueueCron cron -> Cloud Run Job gate-keeper-auditor-queue (* * * * *).

14.4 Coordinator (district / block / village)

- /coordinator — field-ops home with the staff list, allocations and reporting dashboard of their geo scope.
- /coordinator/profile — coordinator profile editor.
- Coordinators do not see the global admin nav — visibleFor in MainDashboardNav filters to / coordinator only.

14.5 Farmer — full route map with data-source verdicts

#	Route	Verdict	Source of truth
1	/farmer	PARTIAL	Aggregates below. Banner copy reflects the mixed boundary.
2	/farmer/prices	REAL	Agmarknet + eNAM MCP -> marketPriceSnapshots -> REST.
3	/farmer/buyers	PARTIAL	Real MongoDB persistence, seeded catalogue

Two production-critical gaps remain explicitly NOT magic-ed away:

1. Auth bypass (was: x-demo-farmer-id). Status: now resolved in this edition (see FARMER_SYSTEM_AUTH_PERSISTENCE_HARDENING.md).
2. Buyer KYC. Status: not in scope. The buyer directory is seeded; KYC integration is a follow-up.

15. Background Jobs — Legacy + Cloud Run Jobs

The system uses node-cron inside the backend for some jobs and Google Cloud Run Jobs (driven by Cloud Scheduler) for everything that should not be tied to a single long-running instance. The GitHub workflow `.github/workflows/cloudrun-jobs-deployment.yml` deploys both new and migrated jobs.

Job (cron syntax)	Status	How it runs
backup-db (0 8,19 * * *)	' Migrated	Cloud Run Job + Cloud Scheduler
gate-keeper-auditor-queue (* * * * *)	' Migrated	Cloud Run Job + Cloud Scheduler
lgd-sync (0 3 * * 0, IST)	' Net-new	Cloud Run Job, weekly on Sunday 03:00
moderator-queue (every 1 min)	& Legacy	node-cron (in-process) — pending migration
time-bound-reallocate (1 min)	& Legacy	node-cron — pending
question-status (1 min)	& Legacy	node-cron — pending
agent-status-cleanup (1 min)	& Legacy	node-cron — pending
daily-report (twice daily)	& Legacy	node-cron — pending
notification-delete (02:00 daily)	& Legacy	node-cron — pending

15.1 Cloud Run Jobs deployment pipeline

- docs/cloudrun-jobs-setup.md is the step-by-step runbook for the one-time GCP setup.
- Each Job gets its own empty service account by least privilege (most Jobs only need Mongo access).
- Per-Job secrets are injected as env vars; GitHub Actions secrets/Variables are the source of truth.
- Manual smoke-test path: `gcloud run jobs execute <name> --region=<region> --wait` then check logs and the bucket.

16. Deployment

16.1 Local dev with Docker Compose

- docker-compose.yml — bring up Postgres, Redis and the agent API.
- docker-compose.mcp.yml — bring up the MCP tool servers.
- docker-compose.app.yml — full app stack (frontend nginx + backend + agent).
- aeagra up / aeagra dev for the agent's hot-reload loop.

16.2 Pre-built images — Portainer / VM

- docker-compose.pull.yml — pulls pre-built images.
- docker-compose.pull-build.yml — pull-then-build for hybrid environments.
- Set stack secrets in Portainer; never override DATABASE_URL with a host port.

16.3 Firebase Hosting

- Site names: agriseva-prod and agriseva-staging-594 (registered, cannot rename without new sites).
- firebase.json — hosting config with SPA fallback to index.html.
- .firebaserc — multi-project registry.

16.4 Production readiness checklist (from DEPLOYMENT_READINESS.md)

#	Blocker	What's needed
1	Firebase credentials not configured	Project ID + web config + service-account JSON
2	MongoDB Atlas URI missing	DB_URL connection string + DB_NAME
3	Anthropic API key missing	ANTHROPIC_API_KEY
4	MiniMax not configured	MINIMAX_BASE_URL + MINIMAX_API_KEY
5	Embedding endpoint missing	GOLDEN_EMBEDDING_ENDPOINT
6	REMOTE_IP not set	Tailscale IP of the self-hosted services node
7	MCP servers not running	All FastMCP services reachable
8	Docker images not built	Build locally or push to Docker Hub

17. CI / CD

- `.github/workflows/build_and_deploy_reviewer.yml` — builds and deploys the reviewer backend.
- `.github/workflows/cloudrun-jobs-deployment.yml` — deploys / updates / executes Cloud Run Jobs (with `target_job` input).
- `.firebaserc` + `firebase.json` — Firebase Hosting targets.
- Scalar-generated OpenAPI surfaces the backend contract for downstream consumers (frontend client, external integrators).

18. Testing

18.1 Frontend

- Vitest for unit + component tests; pnpm run test.
- @testing-library/react for component testing.
- MSW for request mocking in dev + tests.

18.2 Backend

- Vitest with Vitest UI; pnpm run test (UI), pnpm run test:ci (coverage + HTML report).
- Per-module tests under each module's tests/ folder.
- Recent additions:
 - - tests/transaction/TransactionAuth.test.ts — 9/9 (auth, ownership, header-forgery).
 - - tests/user/UserProfile.test.ts — 5/5 (GET, PATCH, round-trip, dedup, 401).
 - - tests/notification/NotificationFlow.test.ts — 4/4 (envelope, mark-all-read, mark-single-read, 401).
 - - tests/marketIntelligence/MarketPricesController.test.ts — 17/17 (Agmarknet path).

18.3 AI agent

- pytest under ai/.
- tests/api/run_api_contracts.py — API contract fuzz tests.
- tests/mcp/mcp_connectivity.py — connectivity probes for every MCP URL.
- evaluation/ harness (deepeval + langsmith + custom plan-based scoring).

19. Observability, Security, Compliance

19.1 Observability

- Sentry for backend error + performance tracking. sentry:sourcemaps script uploads release artifacts.
- Module-level logger via utils/logDetails.ts (PII-safe).
- LangGraph thread logs persisted to MongoDB (thread_log_mongo.py) for every agent run.
- Vertex AI / LangSmith integration in evaluation/ when keys are provided.

19.2 Security model

- All secrets live in .env (gitignored). .env.example files contain placeholders only.
- Per-request Firebase JWT verification — never trust a client-supplied header.
- CORS allow-list driven by APP_ORIGINS.
- Service-to-service trust via INTERNAL_API_KEY.
- Containerised services are reachable only via Tailscale or the cluster network.
- All write paths emit an audit trail entry (auditTrails collection).

19.3 Compliance & disclaimers

- PrivacyPage + TermsPage explain that buyer KYC is not yet integrated and payments are records only.
- FarmerLayout demo banner now reflects the truthful live-persisted-mixed boundary.
- Agmarknet + eNAM data attribution displayed on farmer price UI.

20. Recent Work — Sidebar Toggleable Everywhere

The user reported that the global sidebar was not toggleable on small viewports, that the farmer dashboard wasn't the first item for farmer users, and that several routes referenced by the nav didn't exist. This edition ships the fixes.

20.1 What changed

File	Before	After
MainDashboardSidebar.tsx	Persistent-only on desktop; unmounted on...	Dual-mode: drawer slide-in on mobile
MainDashboardShell.tsx	Single state for the desktop sidebar.	Two-state sidebar model (drawer open/closed)
MainDashboardHeader.tsx	Static hamburger on small screens.	Dual-mode hamburger that toggles drawer
MainDashboardNav.tsx	Single mid-list dashboard entry; multiple...	Farmer Dashboard is FIRST for roles
MainDashboardNav.tsx:118	TypeScript null-check failure: (u) =>...	Fixed to (u) => !!u &&...

20.2 Verification steps run

1. Read all four modified components to confirm the intended final state.
2. Listed every route referenced by NAV_ITEMS and confirmed each one exists as frontend/src/routes/_authenticated.<name>.tsx.
3. Ran tsc --noEmit — 0 errors in any MainDashboard*.tsx.
4. Bypassed pnpm 10 verify-deps gate via frontend/.npmrc -> verify-deps-before-run=false.
5. Started Vite dev server on 127.0.0.1:5173 — HTTP 200 on /, /farmer.
6. HMR served each modified component without warnings.
7. Documented for the owner to complete a manual browser-verification pass.

20.3 Outstanding optional items

- Add dashboard.openMenu | closeMenu | expandNav | collapseNav | globalNav | tabs* keys to all 22 locale files (batch follow-up).
- Add visual regression screenshots of the three viewports to docs/.

21. Done — Complete Inventory

Area	Workstream	Status	Evidence
Frontend	Farmer Dashboard surface (13 routes)	' Complete	frontend/src/features/farmerDashb
Frontend	Coordinator console	' Complete	routes/_authenticated.coordinator.
Frontend	PAE-Expert authoring	' Complete	routes/_authenticated.pae-expert.
Frontend	Moderator / Auditor / Admin consoles	' Complete	routes/_authenticated.{home,audi
Frontend	Per-user profile + history	' Complete	routes/_authenticated.user.\$userlo
Frontend	Responsive shell with toggleable sidebar	' Complete	MainDashboardShell/Sidebar/Hea
Frontend	22-language i18n	' Complete	frontend/src/locales/*
Backend	Auth + persistence hardening	' Complete	FARMER_SYSTEM_AUTH_PERS
Backend	x-demo-farmer-id removed	' Complete	Same
Backend	Notification fan-out for transaction events	' Complete	NotificationService.saveTheNotific
Backend	OpenAPI / Scalar doc generation	' Complete	backend/scripts/generate-openapi
Backend	Cloud Run Jobs migration for backup + queue	' Complete	.github/workflows/cloudrun-jobs-..
AI	Supervisor + 6 subagents	' Complete	ai/agriseva/agents/*
AI	Planner orchestrator (default)	' Complete	planner.py + plan_executor.py
AI	Translation pipeline (Sarvam)	' Complete	translate_answer.py + language.py
AI	WhatsApp + Plivo channel wiring	' Complete	backend chatbot/ + plivo/
MCP	11 tool servers	' Complete	mcp/mcp_containers/
MCP	Agmarknet + eNAM mandi scraping	' Complete	marketIntelligence/ + agmarknet-m
Infra	Secret hygiene (.env vs .env.example)	' Complete	DEPLOYMENT_READINESS.md
Infra	CORS driven by APP_ORIGINS	' Complete	shared/middleware/corsHandler.ts
Infra	CI build + deploy workflows	' Complete	.github/workflows/*
Doc	README, DEPLOYMENT_SETUP,...	' Complete	Top-level .md files
Doc	Farmer audit + hardening report	' Complete	FARMER_DASHBOARD_DATA_A
Doc	This Project Bible	' Complete	docs/project-bible.pdf

22. Pending — Backlog, Blockers, Roadmap

This is the explicit, honest list of work that is NOT done. Every row has an owner, a blocker, and the next concrete step that should be taken. The owner is the project owner unless stated otherwise.

#	Item	Category	Blocker / What's needed	N
1	Real payment gateway	Farmer flow	PSP choice (Razorpay / Stripe / Cashfree) + ...	P
2	Buyer KYC integration	Farmer flow	KYC vendor (Setu / Signzy) + agreement	V
3	Real warehouse / cold-storage API	Farmer flow	Partner warehouse API	E
4	Logistics provider API integration	Farmer flow	Logistics aggregator (Shiprocket / Delhivery)	P
5	Reduce LLM involvement for matched GDB...	AI perf	Decision-tree shortcut in plan_executor	A
6	Propagate location to subagents	AI perf	Subagent prompts + context plumbing	P
7	STT coverage for rarer languages	AI / voice	Sarvam language model versions + per-...	A
8	Migrate remaining 6 legacy cron jobs to Clou...	Infra	Same pattern as backup-db / gate-keeper-...	A
9	Add dashboard.tabs* + sidebar keys to 22...	i18n	Batch translation pass	E
10	Admin moderation policy engine (audit + ...	Ops	Product rule set + UI surface	S
11	Visual regression tests for responsive shell	Frontend QA	Playwright snapshots at 360 / 768 / 1280 / ...	V
12	End-to-end test against real Firebase + Atlas	QA	Test infra with private sandbox cluster	S
13	Coordinator audit + reporting enhancements	Field ops	Aggregation pipeline for district/block...	S
14	Public landing page A/B test	Growth	Marketing experiment harness	P
15	Mobile app (React Native)	Client expand	Bundle budget + design	D

23. Tech Debt & Known Limitations

#	Limitation	Layer	Mitigation
1	name field on farmer profile form is not persisted (lives on...	Frontend	Follow-up: add PATCH /users/me/name ne
2	Seed data contains the literal 'demo-farmer-uid' string in...	Backend	Cosmetic; not used as auth trust. Safe to r
3	Notification fan-out is fire-and-forget (no transactional...	Backend	Acceptable: event-driven. Documented.
4	vitest unit tests mock Firebase tokens; real e2e requires...	QA	Tracked under pending #12.
5	558 pre-existing TypeScript errors in unrelated files...	Code health	Out of scope for this edition; filter out for n
6	5xx on Agmarknet upstream will surface as prices error i...	MCP	Backed by marketPriceSnapshots so the e
7	PNG previews of the responsive shell not committed to...	Docs	Will be added after manual browser-verific

24. Repository Metric Snapshot

Counts derived from the working tree at the time of this PDF. Useful for sizing decisions.

Tree	Files
Frontend routes	28
Frontend feature areas	13
Frontend locale packs	22
Backend modules	23
AI subagent / synthesis files	~40 Python files under agriseva/agents/
MCP tool servers	11
Top-level project docs	README + DEPLOYMENT_SETUP + ...
Cloud Run Jobs (migrated)	2 (+ 1 net-new lgd-sync)
Cloud Run Jobs (legacy cron to migrate)	6
Farmer dashboard routes	13
i18n languages	22
Test suites added (this edition)	3 new files, 18 new tests
Lines touched by auth hardening	+876 / -177
Sidebar edition files modified	5 (4 components + .npmrc)

25. Glossary

Term	Meaning
ACC agent	Acceptance / correctness / compliance agent — human-in-the-loop reviewer
Agmarknet	Government of India's agricultural commodity prices portal (primary mandi source)
aegra	The LangGraph dev/runtime harness used to start the FastAPI agent
eNAM	Electronic National Agriculture Market — secondary mandi source
FPO	Farmer Producer Organisation — a collective that acts as a buyer
GDB	Golden Dataset — expert-verified question/answer pairs
Golden Embedding	BAAI/bge-large-en-v1.5 embedding model used for vector retrieval
KVK	Krishi Vigyan Kendra — agricultural extension centres that publish PoP
LGD	Local Government Directory — states / districts / blocks / villages reference
MCP	Model Context Protocol — standardised tool-calling surface over HTTP
PAE	Project Agricultural Expert — content authorship role
PoP	Package of Practices — crop- and state-specific agricultural guidelines
RAG	Retrieval-Augmented Generation — answers grounded in retrieved context
Reviewer	Human who validates or rewrites an AI-generated answer
Sarvam	Sarvam AI — Indian-language translation + speech service
Subagent	A dedicated LLM ReAct loop responsible for a single domain
Supervisor	The main agent that picks subagents and orchestrates the answer

26. Sign-Off & Maintenance

This PDF is generated programmatically from the source tree and the project's existing documentation. To regenerate after changes:

```
cd C:\Users\saini\OneDrive\Desktop\CSP\AgriSeva-AI
node docs/build-project-pdf.js
# -> docs/project-bible.pdf
```

Recommended cadence: regenerate after any milestone (auth hardening, MCP addition, Cloud Run job migration, locale batch) and re-distribute to reviewers. The build script is intentionally self-contained and idempotent — no external network calls.

For questions about a specific section, open the linked markdown file in the repo or refer to the evidence column in DONE / PENDING tables above.